Report.

Logistics companies often lose time and money when drivers are guided only by distance or past travel times. Important factors like accidents, traffic jams, bad weather, and road closures are usually treated as alerts after they happen, instead of being built into the route planning. This reactive approach causes delays, inefficiency, and higher costs.

We collected a dataset of 5,952 readings (62 days, every 15 minutes). It is used to train and test the models. Since the dataset does not include real routes or distances, we used simulated routes for demonstration until mapping APIs are added.

The project builds an AI system that recommends the fastest and safest routes by:

- Using a regression model to predict traffic volume (vehicles per 15 minutes).
- Using a classification model to label traffic states (low, normal, high, heavy).
- Providing drivers and dispatchers with a chatbot that gives predicts and recommendations in natural language.

1. To make sure the system runs smoothly, the environment must be prepared correctly:
   - Unzip & Open Project
     - Extract *AI Logistics Route Model.zip*.
     - Copy Dataset.csv into the data/ folder.
     - Open the project folder in VS Code.
2. Add Python extension or Jupyter in VS Code.
3. Set up the virtual environment:
     - Create an active virtual environment: On Windows: python -m venv .venv .venv\Scripts\Activate.ps1, On Linux: python3 -m venv .venv source .venv/bin/activate.
     - Install libraries on the terminal: pip install -r requirements.txt
       For instances were activation is blocked, run this: Set-ExecutionPolicy -Scope CurrentUser RemoteSigned
4. Execute the workflow: python src/run_all.py

Designing the Model

- Models used:
    - Regression - predicts traffic volume (vehicles per 15 min).
    - Classification – label traffic state that a driver can easily read.
- Acquired dataset:

- 5,952 rows (62 days × 96 readings).
- No missing values or duplicates.
- Traffic Class balance: normal 60.7%, heavy 19.1%, low 14.0%, high 6.2%.
- Rare events: accidents 5.6%, closures 5.5%.
- Rush hour avg: 167 vehicles vs off-peak 90.

For safety measures, Live counts were excluded from regression to avoid data leakage. Accidents, weather and closures have little statistical impact on volume, so they are applied as explicit penalties in the route-scoring formula instead of being learned by the models.

- Feature Engineering:
  - Time: hour, sine/cosine, day, weekend, rush hour.
  - Hazards: weather severity 0 - 3, accident, closure.
  - History: lags: 15 min, 30 min, 1h, yesterday, rolling mean).
  - Live counts (only for classification to avoid leakage in regression).
- Time Series Analysis:
  - The data shows a strong daily cycle**,** with traffic peaking at 16:00 at rush hour
  - Autocorrelation values:
  - 0.83 with the reading 15 minutes earlier**.**
  - 0.67 with the reading 1 hour earlier**.**
  - 0.64 with the reading at the same time yesterday**.**

These correlations provide clear evidence for using recent traffic history in the model design.

- Algorithms Tested:
  - Regression: Linear Regression, Random Forest, Gradient Boosting, Neural Network.
  - Classification: Logistic Regression, Decision Tree, Random Forest.
  - LSTM suggested for future work.
- Evaluation Plan:

    - Train/test split by day (48 train, 13 test).
    - TimeSeriesSplit validation.
    - Metrics: MAE, RMSE, R^2 (regression); Accuracy, macro-F1, confusion matrix (classification).
    - Baselines: persistence (regression), most frequent class (classification).
- Route Scoring formula:
    - Adjusted travel time = base time + congestion penalty + accident/weather penalties.
    - Closed roads excluded; lowest adjusted time recommended.
- Chatbot:
    - NLP intent classification (11 intents, TF-IDF + Logistic Regression).
    - Entity extraction via regex/keywords.
    - Integrates model outputs into natural language answers.
    - Speech synthesis and speech recognition to allow easier human interaction.